Дослідження архітектури

Об'єднання FEM Solver та CadEditor

у єдиний проект (Project 3)

Мельник О. С.

Адаптивні методи скінченних елементів

Магістерська робота

2026

Зміст

Розділ 1

Огляд поточних проектів

1.1 Solution1 — FEM Solver

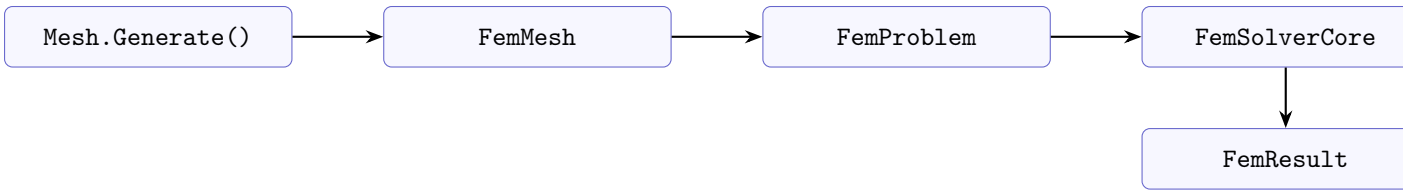
Розташування: `/AdaptiveMethods/Solution1/`

Проект реалізує повний pipeline розв'язування задачі тривимірної лінійної пружності МСЕ на структурованій гексагональній сітці з **20-вузловим серендіп елементом** (H20).

1.1.1 Ключові модулі

Файл	Призначення
`Mesh.cs`	Генерація структурованої сітки: рівномірний поділ паралелепіпеда, підтримка зсувного спотворення `shearXZ`
`FemMesh.cs`	Універсальний формат сітки для солвера: `Positions[N,3]`, `Connectivity[E,20]`
`ShapeFunctions.cs`	20 базисних функцій, передобчислена таблиця похідних `DFIABG[27,3,20]` для 27 точок Гаусса
`ElementStiffness.cs`	Матриця жорсткості елемента $_{60\times60}$, ізопараметрична квадратура
`GlobalAssembly.cs`	Стрічкова збірка глобальної матриці `MG`
`SparseAssembly.cs`	Розріджена збірка (CSR) для великих задач
`BoundaryConditions.cs`	Закріплення методом великого числа (penalty)
`GaussSolver.cs`	Прямий LU-розклад для стрічкової системи
`CsrMatrix.cs`	PCG-солвер з діагональним передобумовлювачем
`StressComputation.cs`	Усереднення напружень по вузлах, Von Mises
`FemProblem.cs`	Постановка задачі: сітка + матеріал + ГУ + навантаження
`FemSolverCore.cs`	Диспетчер: $\leq$ 2000 DOF $\rightarrow$ Gauss; більше $\rightarrow$ PCG

1.1.2 Потік даних



1.1.3 Формат вузлів 20-вузлового елемента

Локальний індекс	Тип	Грань	Координата γ
0–7	кути	—	—
8–11	середини ребер	нижня грань	$\gamma = -1$
12–15	середини ребер	верхня грань	$\gamma = +1$
16–19	середини ребер	вертикальні	$\gamma = 0$

Правило допустимості вузла: позиція $(i_x,\, i_y,\, i_z)$ у регулярній сітці є вузлом тоді і тільки тоді, коли

$$(i_x \bmod 2) + (i_y \bmod 2) + (i_z \bmod 2) \;\le\; 1.$$

1.2 MastersDegreeProject — CadEditor

Розташування: `/MastersDegreeProject/CadEditor/`

Проект реалізує **редактор сітки** з підтримкою:

- збірки довільної форми з кількох батьківських шестигранників,
- адаптивного локального згущення (шаблони T1/T2/T4),
- топологічних зв'язків між елементами (6 сусідів),
- перевірки конформності після згущення.

1.2.1 Ключові класи MeshKernel

Клас	Призначення
`HexElement`	20 вузлів: 8 кутів (`Nodes[]`) + 12 середин (`Midpoints[]`); батько/діти для дерева рефайнменту; 6 покажчиків на сусідів
`MeshNode3D`	Координати (x, y, z); прапори `IsBoundary`, `IsConstraint`, `MidpointOf`
`Mesh3D`	Основний контейнер: автозлиття вузлів за координатами (snap 10^{-9}), зворотний індекс вузол→елементи
`IsoparametricMapper`	Відображення $(\xi, \eta, \zeta) \in [-1, 1]^3 \to$ фізичні координати через 20 базисних функцій H20
`ParametricGridBuilder`	Збірка конформної сітки з набору цілочисельних комірок
`LocalRefinement3D`	Оркестратор адаптивного згущення: Mark-Apply-Expand
`RefinementTemplateService`	Класифікація елемента за кількістю відмічених кутів → T1/T2/T4
`T1Template3D`	Кутовий шаблон: $3n - 2$ дочірніх елементів
`T2Template3D`	Реберний шаблон: $2n - 1$ дочірніх елементів
`T4Template3D`	Граневий шаблон: n дочірніх елементів-плит
`NodePropagation3D`	Прокладка міток через дітей вже розбитого батька
`Mesh3D.Validate()`	5 перевірок конформності та узгодженості
`MeshSerializer`	JSON (версія 2) сериалізація/десеріалізація

1.2.2 Повне дерево рішень рефайнменту

`RefinementTemplateService.Decide(hex)` повертає одну з 7 дій:

Дія	Умова (hex)	Результат	Реалізація
`Skip`	0 або 8 відмічених	нічого	—
`T1`	лист, 1 кут	$3n-2$ дітей	`T1Template3D.Apply`
`T2`	лист, 2 кути (ребро)	$2n-1$ дітей	`T2Template3D.Apply`
`T4`	лист, 4 кути (грань)	n дітей	`T4Template3D.Apply`
`PropagateT1`	не-лист, `Template==T1`	мітки → діти	`NodePropagation3D`
`PropagateT2`	не-лист, `Template==T2`	мітки → діти	`NodePropagation3D`
`PropagateT4`	не-лист, `Template==T4`	мітки → діти	`CascadeT4`

Особливий випадок 2 міток

Якщо два відмічених кути **не** лежать на спільному ребрі, але лежать на спільній грані (діагональ грані) — вирішення T4 для тієї грані. Якщо вони утворюють просторову діагональ тіла — `Skip` (алгоритмічно недопустима конфігурація).

1.2.3 NodePropagation3D — рефайнмент вже розбитого батька

Ключова ситуація

Батько вже розбитий шаблоном T1 або T2 (`hex.IsLeaf == false`). Нові мітки приходять на його зовнішні вузли. **Наївне рішення** — застосувати T2 тільки до зовнішньої дитини. **Правильне рішення** — NodePropagation поширює той самий шаблон по всьому ланцюжку дітей від зовнішньої оболонки аж до кутового кубика, що забезпечує рівномірне згущення і кращу якість сітки.

Ключова ідея поширення (верифіковано по коду) T1-батько з $n = 2$ має 4 дітей: outer X-slab, outer Y-slab, outer Z-slab, innermost cube. Рівні T1-шаблону побудовані як:

$$\text{levels}[i][j] = \mathbf{x}_c + \frac{n-i}{n}\big(\mathbf{x}_{c\oplus j} - \mathbf{x}_c\big), \quad i = 0\ldots n,\ j = 0\ldots 7.$$

При `PropagateFromT1` з n_1=2 і новою міткою v_k:

ℓ	$s_\ell = \frac{n_1-\ell}{n_1}$	Позиція $\mathbf{p}_\ell$	Яку дитину знаходить	Нова мітка на дитині
0	1.0	$\mathbf{x}_{v_k}$ (зовні)	outer X-slab (має `outer`$[v_k]$ як кут)	зовнішній кут
1	0.5	midpoint(c, v_k)	**outer X-slab** (має `inner`$[v_k]$ як кут) **і innermost cube** (має levels$[1][k]$ як кут)	внутрішній кут єдиний кут

Після двох проходів:

- outer X-slab отримав **2 мітки** на ребрі (`outer`$[v_k]$ та `inner`$[v_k]$) ⇒ `ApplyTemplatesToMarked` обирає **T2**.
- innermost cube отримав **1 мітку** ⇒ **T1**.

Алгоритм коректний — логіка верифікована по коду

Зовнішня оболонка отримує T2 (плавний перехід вздовж ребра), кутовий кубик — T1. Ланцюжок безперервний.

Стан тестового покриття

Протестовано (є xUnit-тести):

- `Decide()` повертає `PropagateT1` / `PropagateT2` / `PropagateT4` для вже розбитих батьків.

НЕ протестовано — пряма прогалина:

- `ApplyDecision` з дією `PropagateT1/T2` не виконується в жодному тесті.
- Не перевіряється що outer-slab отримав 2 мітки → T2, innermost cube → T1.
- `mesh.Validate()` після пропагації не перевіряється.

Додатковий ризик: `MarkChildrenAtPosition` шукає збіг з точністю $\varepsilon{=}10^{-9}$. При нецілих координатах батька (параметричне перетворення, великий домен) floating-point drift може призвести до того що дитина **не буде знайдена** і мітка загубиться.

Необхідні тести перед інтеграцією з FEM

1. T1 батько ($n{=}2$), нова мітка на v_6 (протилежний кут): перевірити що outer-slab має Template T2, innermost cube — T1, `mesh.Validate()` проходить.
2. T2 батько ($n{=}2$), нова мітка на вузлі, що не лежить на вихідному ребрі: перевірити що трубка propagated коректно.
3. Нецілі координати (наприклад домен $[0, 3.7]^3$): перевірити що `MarkChildrenAtPosition` знаходить всі рівні.

PropagateFromT1$(P,\ \{v_k\},\ n_2)$ — **формально** $n_1 = \lfloor(\texttt{Children.Length} + 2)/3\rfloor$. Для кожного v_k і кожного рівня $\ell = 0 \dots n_1 - 1$:

$$\mathbf{p}_\ell = \mathbf{x}_c + \frac{n_1 - \ell}{n_1}\left(\mathbf{x}_{v_k} - \mathbf{x}_c\right).$$

Відмітити **всіх** дітей, у яких є кут в $\mathbf{p}_\ell$ (один і той самий дочірній елемент може отримати мітки від кількох рівнів одночасно — це навмисно: формує T2 там де потрібен перехід).

PropagateFromT2$(P,\ \{v_k\},\ n_2)$ $n_1 = \lfloor(\texttt{Children.Length}{+}1)/2\rfloor$. Для кожного v_k: знайти у `T2Template3D.EdgeMap` запис з `To` $= k$, взяти `From` як початкову точку трубки та аналогічно прокласти n_1 позицій вздовж трубки від зовнішнього кінця до внутрішнього.

CascadeT4$(P,\ n)$ Знайти всі грані P з усіма 4 відміченими кутами. Передати ті самі мітки всім дітям для кожної такої грані і рекурсивно викликати `Apply`.

ApplyTemplatesToMarked — класифікація дітей після поширення

- 1 мітка на кут → T1
- 2 мітки на ребро → T2 (ключовий випадок: дає безперервний перехід)
- 2 мітки — діагональ грані або 4 мітки — грань → T4
- Інше → T1 з першою міткою (fallback)

Розділ 2

Ключове відкриття: сумісність форматів

Структура 20-вузлового елемента збігається в обох проектах

Порядок вузлів у H20 елементі є **ідентичним** між CadEditor (`HexElement`) та FemSolver (`NT[e, 0..19]`). Адаптер зводиться до простої перенумерації `Guid` → `int`.

2.1 Відповідність вузлів

FemSolver	Вузли	CadEditor	Ребра
`NT[e, 0..7]`	кути	`hex.Nodes[0..7]`	—
`NT[e, 8..11]`	нижня грань	`hex.Midpoints[0..3]`	$(0,1),(1,2),(2,3),(3,0)$
`NT[e,12..15]`	верхня грань	`hex.Midpoints[4..7]`	$(4,5),(5,6),(6,7),(7,4)$
`NT[e,16..19]`	вертикалі	`hex.Midpoints[8..11]`	$(0,4),(1,5),(2,6),(3,7)$

Таким чином, для кожного листового елемента:

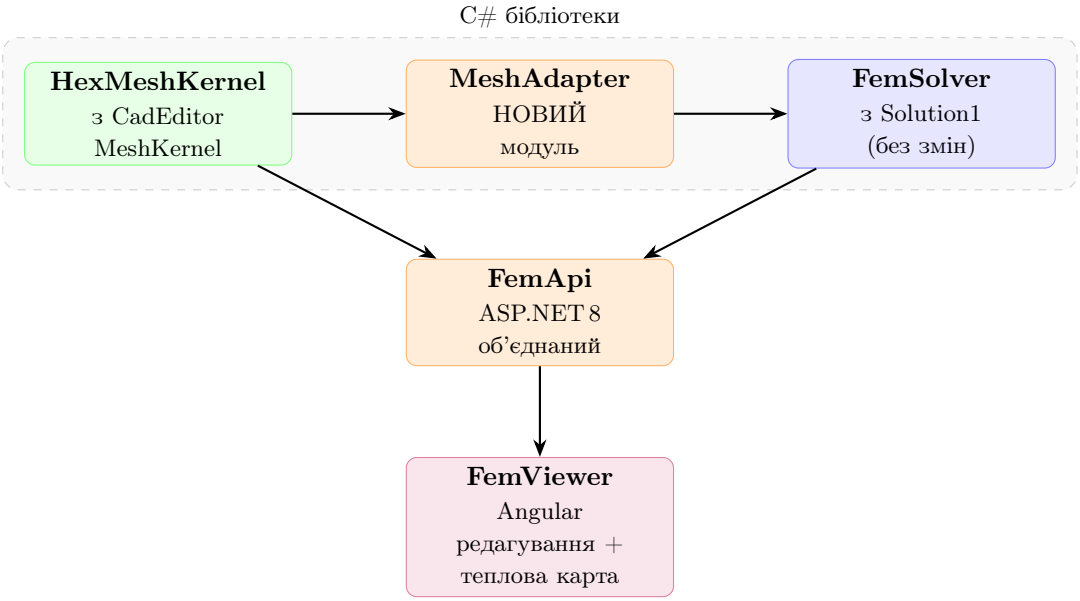
$$\texttt{Connectivity}[e,\, i] = \begin{cases} \text{nodeMap}[\texttt{hex.Nodes}[i]] & i = 0, \ldots, 7 \\ \text{nodeMap}[\texttt{hex.Midpoints}[i-8]] & i = 8, \ldots, 19 \end{cases}$$

Розділ 3

Архітектура Project 3

3.1 Структура рішення

Project 3 є єдиним .NET 8 solution з чотирма C# проектами та одним Angular фронтендом:



Проект	Джерело	Зміни
HexMeshKernel	CadEditor.MeshKernel	мінімальні (очищення)
FemSolver	Solution1/FemSolver	**нуль** — без жодних змін
MeshAdapter	новий	300 рядків нового коду
FemApi	об'єднання обох API	новий endpoint `/fem/solve`
FemViewer	Solution1/FemViewer	додати режим редагування

3.2 Модуль MeshAdapter

Це **єдиний новий код**, що з'єднує два проекти. Складається з трьох компонентів.

3.2.1 MeshExporter — Mesh3D → FemMesh

```
public static FemMesh ExportToFemMesh(IMesh3D mesh)
{
    var leaves = mesh.GetLeafHexes();

```

```
    // 1. Унікальнанумерація : Guid -> int
    var nodeMap = new Dictionary<Guid, int>();
    int idx = 0;
    foreach (var hex in leaves)
        foreach (var id in hex.Nodes.Concat(hex.Midpoints))
            if (!nodeMap.ContainsKey(id)) nodeMap[id] = idx++;

    // 2. Масивкоординат
    var pos = new double[idx, 3];
    foreach (var (id, i) in nodeMap)
    {
        var n = mesh.GetNode(id)!;
        pos[i, 0] = n.X;  pos[i, 1] = n.Y;  pos[i, 2] = n.Z;
    }

    // 3. Таблицязвязності ' порядок( вузліввжезбігається  !)
    var conn = new int[leaves.Count, 20];
    int e = 0;
    foreach (var hex in leaves)
    {
        for (int i = 0; i < 8;  i++) conn[e, i]     = nodeMap[hex.Nodes[i]];
        for (int i = 0; i < 12; i++) conn[e, 8 + i] = nodeMap[hex.Midpoints[i]];
        e++;
    }

    return new FemMesh {
        NodeCount    = idx,
        ElementCount = leaves.Count,
        Positions    = pos,
        Connectivity = conn
    };
}
```

Лістинг 3.1: Ядро адаптера

3.2.2 BCTranslator — граничні умови

Транслює поверхневі мітки з `Mesh3D` у формат `FemProblem`:

CadEditor (вхід)	**FemSolver (вихід)**
`node.IsConstraint == true`	`int[] FixedDofs`
Мічені грані з тиском p	`double[] LoadVector` через інтеграл по грані
`node.IsBoundary`	фільтр для ідентифікації зовнішніх вузлів

3.2.3 ResultMapper — FemResult → Mesh3D

Після розв'язання: `VonMises[nodeIndex]` → словник `Guid` → `double` для відображення теплової мапи на тій самій `Mesh3D`, яку редагував користувач.

Розділ 4

Головна технічна проблема: конформність після рефайнменту

4.1 Суть проблеми

Висячі вузли

Після локального згущення (наприклад, тільки деякі батьківські шестигранники розбито шаблоном T4) на межі між *дрібними* та *грубими* елементами виникають **висячі вузли** (hanging nodes): вузли дрібного елемента, що лежать на ребрах або гранях грубого елемента, але не є його власними вузлами.

У МСЕ висячий вузол порушує C^0-неперервність поля переміщень і дає *некоректний* розв'язок, якщо не вжити спеціальних заходів.

4.2 Два підходи

Підхід	Переваги	Недоліки
Propagation — поширити рефайнмент на сусідів до повного усунення висячих вузлів	Математично просто; FemSolver не змінюється; вже є `LocalRefinement3D`	Більше елементів ніж потрібно; можливий каскад
Constraint equations — додати рівняння зв'язку $u_h = \sum_i \varphi_i\, u_i$	Мінімальна кількість елементів; точне згущення	Складніша збірка СЛАР; треба модифікувати `SparseAssembly`

Рекомендація для Фази 1

Використати **Propagation**: перед експортом у `MeshExporter` автоматично поширити рефайнмент на всіх сусідів висячих вузлів через `LinkLeafNeighbors()` та `Validate()`. На другому етапі, коли базовий pipeline підтверджений, перейти до constraint equations для мінімізації DOF.

4.3 Перевірка конформності

`Mesh3D.Validate()` вже перевіряє:

1. дублікати вузлів за координатами,
2. узгодженість `_nodeIndex` з `_nodes`,

3. коректність посилань вузлів у `Nodes[]` та `Midpoints[]`,

4. узгодженість зворотного індексу `_nodeToHexes`,

5. узгодженість дерева батько↔діти.

Після виклику `LinkLeafNeighbors()` посилання на листових сусідів оновлені; **висячі вузли ідентифікуються як листові вузли без відповідного сусіда** на граневій стороні.

Розділ 5

Покроковий план реалізації

5.1 Фаза 1 — Базовий адаптер (без рефайнменту)

1. Створити `MeshAdapter` з `MeshExporter.ExportToFemMesh`.
2. Перевірити: побудувати просту $1 \times 1 \times 1$ сітку в CadEditor, експортувати та порівняти результат солвера з Solution1.
3. Реалізувати `ResultMapper`: `int[]` → `Guid`.
4. Відобразити Von Mises у FemViewer на сітці CadEditor.

5.2 Фаза 2 — Граничні умови та навантаження

1. Додати до `HexElement` або до API можливість мітити грані як закріплені / навантажені.
2. Реалізувати `BCTranslator`: закріплені грані → `int[] FixedDofs`; розподілене навантаження → `double[] LoadVector` через інтегрування по поверхні.
3. Тест: консольний брус під рівномірним тиском.

5.3 Фаза 3 — Конформний рефайнмент

1. Реалізувати функцію визначення висячих вузлів після `LocalRefinement3D.Run()`.
2. Реалізувати автоматичне propagation до конформної сітки.
3. Додати до `MeshExporter` виклик перевірки перед побудовою `FemMesh`.
4. Тест: L-подібна область з неоднорідним рефайнментом у зоні концентрації напружень.

5.4 Фаза 4 — Параметричні батьківські шестигранники

1. Підтримка нерівномірних (кривих) батьківських гексів через `IsoparametricMapper` при інтерактивному переміщенні вузлів.
2. Всі дочірні вузли перераховуються через параметричне відображення.
3. Тест: циліндричний сегмент.

Розділ 6

Зведена таблиця: що є і що треба збудувати

Функціонал	**Статус**	**Джерело / Дія**
20-вузловий H20 елемент	**Є**	обидва проекти
Рівномірна структурована сітка	**Є**	Solution1/`Mesh.cs`
Збірка з кількох шестигранників	**Є**	CadEditor/`Mesh3D.cs`
Ізопараметрична квадратура	**Є**	Solution1/`ElementStiffness.cs`
Адаптивне локальне згущення	**Є**	CadEditor/`LocalRefinement3D.cs`
Шаблони T1, T2, T4	**Є**	CadEditor
Перевірка конформності	**Частково**	`Validate()` є; propagation потрібен
Розріджений PCG солвер	**Є**	Solution1/`SparseAssembly.cs`
Теплова карта Von Mises	**Є**	Solution1/`HeatMap.cs`
Адаптер Mesh3D → FemMesh	**Потрібен**	новий `MeshExporter`
Транслятор граничних умов	**Потрібен**	новий `BCTranslator`
Зворотній маппінг результатів	**Потрібен**	новий `ResultMapper`
Обробка висячих вузлів	**Потрібен**	propagation або constraint eq.
Об'єднаний API endpoint	**Потрібен**	`POST /fem/solve`
Редагування + heatmap в UI	**Потрібен**	доповнення FemViewer

Висновок

Обидва проекти використовують **H20 серендіп елемент з ідентичним порядком вузлів**, що робить їх інтеграцію технічно прямолінійною. Весь новий код зосереджений в одному модулі — **MeshAdapter** (300 рядків) — і вирішенні конформності після локального рефайнменту.

Ключові переваги об'єднаного проекту:

- **Довільна геометрія**: кілька параметричних батьківських шестигранників замість одного паралелепіпеда;
- **Адаптивне згущення**: концентрація вузлів тільки там, де висока градієнт напружень;
- **Ефективність**: набагато менше DOF ніж при рівномірній сітці тієї ж точності;
- **Модульність**: FemSolver залишається незмінним і може використовуватись незалежно.

Архітектурний принцип: `Mesh3D` $\xrightarrow{\text{MeshAdapter}}$ `FemMesh` $\xrightarrow{\text{FemSolverCore}}$ `FemResult` $\xrightarrow{\text{ResultMapper}}$ `Mesh3D (з кольорами)`.